

ShopSphere

S3 - Category Management and Navigation

Admin Portal

Platform-Specific Sprint Planning Document

This document intentionally defines only the behavior and delivery scope of the selected platform. It does not mix implementation details from the other platforms.

1. Sprint Definition

Sprint: S3 - Category Management and Navigation

Platform: Admin Portal

Application: ShopSphere

Objective: Implement only the admin-web behavior relevant to Category Management and Navigation; do not implement customer behavior in this document.

This sprint is complete only when the stated use cases, acceptance criteria and tests pass for this platform.

2. Business Use Cases

ID	Use Case	Actor	Precondition	Main Flow	Expected Result
UC-01	Create category	Admin	Authorized role	Enter category metadata, save	Category becomes available according to status.
UC-02	Browse categories	Customer	Category exists and is active	Open category navigation	Active hierarchy is displayed.

3. Detailed Implementation Plan

1. Implement category tree/list, create, edit, activate/deactivate and safe-delete.
2. Validate duplicate name/slug and hierarchy rules.

4. Data / State / Contract Scope

- No direct database changes. Data is accessed through backend APIs.

API / Integration Contract

- Consume the corresponding secured /api/v1 admin/customer operational APIs defined by the backend sprint document.

5. Rules and Expected Behavior

- The platform must rely on the backend as the source of truth for business rules.
- Do not expose stack traces, secrets or internal implementation details to end users.
- Handle loading, empty, validation and error states explicitly.
- Requests that can be retried must be safe from duplicate side effects where the feature requires idempotency.
- Ownership and authorization must be checked server-side for protected data.

6. Acceptance Criteria

- Only authorized admin roles can access the feature.
- Admin UI reflects server state and permission boundaries.
- Validation, loading, empty, error and confirmation states are implemented where relevant.
- No customer UI behavior is defined in this document.

7. Test Cases

TC	Scenario	Expected Result
TC-S3-01	Duplicate slug	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S3-02	Self-parent	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S3-03	Circular hierarchy	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S3-04	Inactive parent	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.

TC	Scenario	Expected Result
TC-S3-05	Delete category with products	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S3-06	Empty category	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S3-07	API timeout	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.

8. Sprint Exit Checklist

- Implementation completed only for this platform.
- All primary and negative use cases pass.
- API contracts are implemented/consumed as defined.
- Validation and authorization tests pass.
- Loading/empty/error handling is verified where applicable.
- No critical or blocker defect remains.
- Automated tests pass in CI.
- Documentation and contract changes are committed.

9. Out of Scope for This Document

- Implementation behavior of Backend + DB. That behavior belongs in the separate S3 document for that platform.
- Implementation behavior of Customer Portal. That behavior belongs in the separate S3 document for that platform.

10. Figma Design Implementation Contract

Sprint 3 - Category Management & Navigation | Admin Portal

Purpose: This section makes the approved Figma design the UI/UX source of truth for this sprint while the sprint requirements remain the source of truth for business behavior.

10.1 Mandatory Exact-Screen Selection

- Cursor/Main Agent must use only the Figma frames explicitly mapped for this sprint and platform.
- Do not choose a Figma screen by visual similarity, approximate name, nearby position, or assumption.
- Each required screen must be mapped to an exact Figma frame URL or node ID before UI implementation starts.
- If a required frame URL/node ID is missing, inaccessible, still contains a placeholder, or is ambiguous, Cursor must stop UI implementation and report FIGMA_MAPPING_REQUIRED.
- Do not implement a screen belonging to another sprint merely because it appears related.

10.2 Required Screen Registry

After Figma Make generates and the team approves the design, fill the Exact Figma Frame URL / Node ID column. These identifiers are the mechanism that makes screen selection deterministic.

Screen Key	Required Screen / State	Exact Figma Frame URL / Node ID
S3-ADMIN-01	Category List / Tree	REQUIRED - paste exact approved Figma frame URL or node ID
S3-ADMIN-02	Create Category	REQUIRED - paste exact approved Figma frame URL or node ID
S3-ADMIN-03	Edit Category	REQUIRED - paste exact approved Figma frame URL or node ID
S3-ADMIN-04	Delete / Dependency Warning	REQUIRED - paste exact approved Figma frame URL or node ID

10.3 Cursor Preflight Before Coding

- Read this sprint PDF completely.
- Resolve every required screen in the registry to its exact approved Figma node/frame.
- Inspect the existing codebase and shared component library.
- Create a screen-to-code mapping: Screen Key -> Figma Node -> Route/Screen File -> Existing Components -> Missing Components.
- Inspect exact Figma component variants, design tokens, assets, spacing, typography and interaction states.
- Only after the mapping is complete may UI coding begin.

10.4 Component Reuse / Import Rule

- Figma component -> inspect -> map to existing React Native/React component -> reuse if compatible.
- If a compatible reusable component does not exist, create it in the shared component layer rather than creating a one-off duplicate.
- Use approved Figma assets/icons where technically appropriate.
- Do not blindly paste generated Figma code into production. Adapt the design to the project architecture.
- Do not hardcode arbitrary visual values when approved design tokens/components exist.

10.5 Source-of-Truth Priority

Business behavior -> this Sprint specification. UI/UX -> exact approved Figma frame/node. API request/response -> backend/OpenAPI contract. Database/business invariants -> Backend sprint specification.

10.6 Conflict Rule

- If Figma conflicts with the sprint requirement or backend/API contract, do not silently alter behavior.
- Stop the conflicting implementation portion and report the sprint requirement, Figma behavior, API/backend behavior if relevant, exact conflict and recommended resolution.

10.7 Visual Fidelity and Completion Gate

- Verify exact frame/node used for every required screen.
- Match layout hierarchy, typography, spacing, components, variants, icons/assets and interaction behavior.
- Implement loading, empty, validation, error, disabled and retry states required by this sprint/Figma.
- Customer Portal: verify iOS, Android and responsive Web behavior where applicable.

- Admin Portal: verify desktop/tablet responsive behavior and permission-aware states.
- Connect real APIs required by the sprint; do not replace required integrations with mocks/stubs.
- Run lint, type-check, tests and build.
- Compare implemented UI against the exact approved Figma frame before marking the sprint DONE.
- No unmapped screen may be invented.

10.8 Ready-to-Copy Cursor Instruction

Implement Sprint 3 Admin Portal according to this sprint document. For UI, use ONLY the exact approved Figma frames listed in Section 10.2. Before coding, resolve every required frame URL/node ID and produce a screen-to-code mapping. Never select a frame by similarity or approximate name. If any required mapping is missing or ambiguous, stop and return: FIGMA_MAPPING_REQUIRED: S3 ADMIN. Reuse existing shared components when compatible; create missing reusable components in the shared layer. Connect the real sprint APIs, implement all required states, run tests/build, and visually compare every implemented screen against its exact Figma node before marking DONE.